

Relational vs. NoSQL modeling

COMPLETE LEARNING GUIDE

What You'll Learn

Choosing between a relational database and a NoSQL document store is a modeling decision, not a popularity contest — the two paradigms make different trade-offs around consistency, schema flexibility, and how relationships between data are represented. This guide covers relational normalization and referential integrity through foreign keys, document modeling with the core embed-vs-reference decision that governs most MongoDB schema design, a practical (not purely academic) framing of the CAP theorem, and concrete guidance for choosing between the two — including the reality that most serious production systems end up using more than one data store for different purposes (polyglot persistence) rather than forcing every kind of data into a single database technology.

1. Relational Modeling and Normalization

Normalization organizes relational data to minimize redundancy: instead of repeating a student's name and email on every row of an enrollments table, you store student data once in a students table and reference it by ID from enrollments. Third normal form (3NF) — the practical target for most OLTP schemas — requires that every non-key column depends on the whole primary key and nothing but the key, eliminating both duplicate data and the update anomalies that come with it (e.g., updating a student's email in one row of a denormalized table while forgetting three other rows that still show the old one).

2. Foreign Keys and Referential Integrity

A foreign key constraint declares that a column's value must match an existing value in another table's key column, and the database itself enforces this — it will reject an insert that references a non-existent parent row, and it will act according to a declared ON DELETE policy (CASCADE, SET NULL, RESTRICT) when a referenced row is deleted. This is what prevents 'orphaned' data (an enrollment row pointing to a student_id that no longer exists) at the database layer, rather than relying on every piece of application code to remember to check this manually.

SQL

```
CREATE TABLE students (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL  
);  
  
CREATE TABLE enrollments (  
  id SERIAL PRIMARY KEY,  
  student_id INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,  
  course_id INTEGER NOT NULL REFERENCES courses(id),  
  enrolled_at TIMESTAMP DEFAULT NOW()  
);  
-- Deleting a student automatically removes their enrollments (CASCADE)
```

3. Document Modeling in NoSQL

A document database like MongoDB stores each record as a self-contained, typically JSON-like document, and doesn't enforce a schema across documents in the same collection the way a relational table enforces column types across all rows. This flexibility is well-suited to data with a naturally nested, hierarchical, or evolving shape — a student profile with an arbitrary, growing list of certifications, for instance — where a relational schema would otherwise require several joined tables just to reconstruct one logical record.

JSON

```
{  
  "_id": "665f1a2b3c4d5e6f7a8b9c0d",  
  "name": "Aisha Khan",  
  "email": "aisha@example.com",  
  "enrollments": [  
    { "courseId": "fsd-2026", "enrolledAt": "2026-01-15", "status": "active" }  
  ]  
}
```

4. Embedding vs. Referencing in Document Databases

The single most consequential decision in document modeling is whether related data lives embedded inside the parent document or in a separate collection referenced by ID. Embed when the related data is always accessed together with its parent, is bounded in size, and doesn't need to be queried independently — a student's address, for example. Reference (store an ID and look it up separately, similar to a foreign key) when the related data is large, unbounded, shared across multiple parents, or frequently queried on its own — a student's enrollments across potentially hundreds of courses, for instance, which would otherwise make the student document grow without bound and force rewriting the entire document on every new enrollment.

- MongoDB documents have a hard 16MB size limit — an unbounded embedded array (e.g., logging every login event inside a user document) can eventually hit it.
- Data that needs to be queried or updated independently of its 'parent' (e.g., searching all enrollments across all students) is a strong signal to reference rather than embed.

5. The CAP Theorem in Practice

The CAP theorem states that a distributed data system can only fully guarantee two of three properties during a network partition: Consistency (every read gets the latest write), Availability (every request gets a response, even if not the latest data), and Partition tolerance (the system keeps working despite network failures between nodes). Since network partitions do happen in real distributed systems, partition tolerance isn't really optional — the practical trade-off most engineers actually face day to day is CP vs. AP: does this specific piece of data need to always be perfectly up to date even if that means some requests fail during a partition (CP — typical for financial balances), or is it acceptable to serve slightly stale data to keep the system always responsive (AP — typical for a social media feed or product view count)?

6. When to Choose Relational vs. NoSQL

Relational databases fit best when data has clear, stable structure and relationships, when strong consistency and transactional guarantees across multiple related records matter (a payment and the order it's tied to must both succeed or both fail), and when you'll need to query across relationships in ways you can't fully predict at design time (SQL's flexibility here is a real advantage). Document databases fit best when the schema is naturally variable or evolving, when the primary access pattern is 'fetch one big document by ID' rather than complex cross-entity joins, and when horizontal write scaling across many servers matters more than strict cross-record consistency.

7. Hybrid and Polyglot Persistence

Most non-trivial production systems don't force every kind of data into one database technology — a typical setup might use a relational database (Postgres) for core transactional data (users, orders, payments) where consistency and relationships matter most, alongside a document store or search index for a specific workload it's better suited to (a product catalog with wildly varying attributes per category, or full-text search), and a cache (Redis) for hot, frequently-read data. This 'polyglot persistence' approach trades some operational complexity (more systems to run and keep in sync) for using each tool where it's actually strongest.

Common Pitfalls

- Over-normalizing a read-heavy OLTP schema to the point where common queries require six or seven joins for data that's almost always read together.
- Modeling a document database exactly like a relational one, with many small referenced collections, which defeats the point of embedding and forces application-level joins MongoDB isn't optimized for.

- Embedding an array that can grow unbounded (e.g. an activity log) directly inside a parent document, risking hitting the 16MB document size limit.
- Ignoring referential integrity in a NoSQL store because 'there's no foreign key constraint,' leading to silently orphaned references over time.
- Choosing NoSQL purely because it 'scales better' without actually needing schema flexibility or that specific scaling profile, then fighting the lack of joins and transactions later.
- Not planning for schema evolution in a schemaless database, leading to a collection with wildly inconsistent document shapes across old and new records.

Key Takeaways

- Normalization eliminates redundancy and update anomalies; foreign keys enforce referential integrity at the database layer, not just in application code.
- In document modeling, embed for data always accessed together and bounded in size; reference for large, shared, or independently-queried data.
- CAP theorem trade-offs matter mainly during network partitions — the practical day-to-day question is usually CP (always-correct) vs. AP (always-available).
- Relational fits stable, relationship-heavy data needing strong consistency; document stores fit variable, document-shaped data with simple access patterns.
- Most real production systems use more than one data store, each for the workload it's best suited to, rather than one database for everything.

Learn More

PostgreSQL Docs — Normalization

<https://www.postgresql.org/docs/current/tutorial-normalization.html>

MongoDB — Data Modeling: Embedding vs Referencing

<https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>

MongoDB — Building With Patterns (Schema Design)

<https://www.mongodb.com/developer/products/mongodb/schema-design-anti-patterns-summary/>